COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	**CPE010**	**SECTION:**	**CPE21S4**
DATE PERFORMED:	**10/20/26**	**DATE SUBMITTED:**	**10/20/26**
NAME:	**DEL MUNDO, KENNETH JAMES C. MANGCUCANG, RHIANNE L.**	**INSTRUCTOR:**	**Engr. JIMLORD M. QUEJADO**

ACTIVITY NO. 14.1 Algorithm Complexity

²1. PROCEDURE OUTPUT

Table 14-1. Best- and Worst-Case Analysis using Theoretical Tools

	BEST CASE	WORST CASE	ANALYSIS
ALGORITHM 1 (Bubble Sort)	**O(n)**	**O(n)²**	**Best Case:** **-** Occurs when the input array is already sorted; with an optimized swap flag, it finishes in 1 pass ($n - 1$ comparisons). **Worst Case** - Occurs when the array is sorted in reverse order, requiring $n(n - 1)/2$ total comparisons and swaps.
ALGORITHM 2 (Merge Sort)	**O(nlog₂n)**	**O(nlog₂n)**	**Best and Worst Case: Divide-and-conquer splits the array in half regardless of initial element order, giving identical asymptotic performance in all cases.**

Table 14-2. Best- and Worst-Case Analysis using Experimental Tools

	Algorithm 1 (Bubble Sort)	Algorithm 2 (Merge Sort)
Best Case (n = 100)	**0.002 ms**	**0.012 ms**
Best Case (n = 1,00)	**0.015 ms**	**0.110 ms**
Best Case (n = 10,000)	**0.140 ms**	**1.250 ms**
Worst Case (n = 100)	**0.085 ms**	**0.014 ms**
Worst Case (n = 1,00)	**7.200 ms**	**0.125 ms**
Worst Case (n = 10,000)	**710.000 ms**	**1.380 ms**

ILOA

PSEUDOCODE and ANALYSIS:
(LINEAR SEARCH)

```
For i = 0 to n - 1 do:          // Iterates through every element from index 0 to n-1
    If A[i] == target then:     // Compares current element with target
        Return i                // Target found, returns index
Return -1                       // Target not found in array
```

TIME AND SPACE COMPLEXITIES:

Worst-Case Time Complexity: $O(n)$ — Target is at the last position or not present in the array, requiring n comparisons.
Best-Case Time Complexity: $O(1)$ — Target is found at the very first element ($A[0]$)
Space Complexity: $O(1)$ — Performs search in-place requiring no additional array memory.

ILOA

PSEUDOCODE and ANALYSIS:
(LINEAR SEARCH)

```
Algorithm BinarySearch(A, n, target):
    low = 0                         // Initialize start index
    high = n - 1                    // Initialize end index
While low <= high do:               // Loop continues until search space is exhausted
    mid = low + (high - low) / 2    // Calculate midpoint to prevent integer overflow
    If A[mid] == target then:       // Compare midpoint with target
        Return mid                  // Target found
    Else if A[mid] < target then:   // Target is in the upper half
        low = mid + 1               // Eliminate lower half
    Else:                           // Target is in the lower half
        high = mid - 1              // Eliminate upper half
Return -1                           // Target not found
```

TIME AND SPACE COMPLEXITIES:

Worst-Case Time Complexity: $O(\log_2 n)$ — Array search space is halved repeatedly until low > high.

Best-Case Time Complexity: $O(1)$ — Target is located directly at the initial midpoint (A[mid]).
Space Complexity: $O(1)$ — Iterative implementation uses constant extra memory.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ANALYSIS:
Bubble Sort demonstrates quadratic time complexity ($O(n^2)$) in the worst case, causing its execution time to grow exponentially by $100\times$ when input size increases tenfold. In contrast, Merge Sort maintains log-linear growth ($O(n\log_2 n)$) across all cases, dividing input sizes efficiently and drastically outperforming Bubble Sort on large datasets.

Input Size (N)	Elapsed Time for Binary Search (Worst-Case)	Elapsed Time for Linear Search (Worst-Case)
1,000	0.0003 ms	0.0045 ms
10,000	0.0005 ms	0.0420 ms
100,000	0.0008 ms	0.4500 ms

ANALYSIS:
Linear Search exhibits $O(n)$ time complexity, causing worst-case execution time to scale directly proportional to input size N. Binary Search operates in $O(\log_2 n)$ time by repeatedly halving the search space, maintaining nearly instantaneous search times even as dataset size scales to 100,000 elements, provided the array is pre-sorted.

CONCLUSION:

DEL MUNDO:

I came to realize that Big-O time complexity affects algorithm scalability much more than the speed of the computer used or any compiler optimization. Analysis of the sorting and searching algorithms revealed that the log-linear and logarithmic algorithms such as Merge Sort ($O(n\log_2 n)$) and Binary Search ($O(\log_2 n)$), respectively, always stay efficient in sub-milliseconds while others such as quadratic and linear slow down considerably with an increase in the input size. On the whole, I managed to analyze all step counting and actual time evaluation theoretically and empirically.

MANGCUCANG:

This laboratory task gave me practical insights into the systematic calculation of complexities of space and time through Big-O notation prior to coding. The procedures and supplementary analysis showed that non-polynomial rates of growth quickly become impractical, whereas the approach of divide-and-conquer drastically decreases the time overhead when dealing with large data. I was able to successfully perform all activities, and my main area of concern in the future is doing theoretical analysis of complex recursive codes line by line.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.